FATEC DR. THOMAZ NOVELINO
DESENVOLVIMENTO DE SOFTWARE MULTIPLATAFORMA - LABORATÓRIO DE DESENVOLVIMENTO PARA DISPOSITIVOS MÓVEIS

GABRIEL DA SILVEIRA PESSONI
LÍVIA PORTELA FERREIRA
HUDSON RIBEIRO BÁRBARA JUNIOR

APLICATIVO INTELIGENTE DE CONTROLE FINANCEIRO PESSOAL

FRANCA
2026

GABRIEL DA SILVEIRA PESSONI

LÍVIA PORTELA FERREIRA

HUDSON RIBEIRO BÁRBARA JUNIOR

APLICATIVO INTELIGENTE DE CONTROLE FINANCEIRO PESSOAL

Trabalho apresentado à disciplina Laboratório de Desenvolvimento para Dispositivos Móveis, do curso de Desenvolvimento de Software Multiplataforma da FATEC Dr. Thomaz Novelino, como requisito parcial para obtenção de nota no 5º semestre.

Orientador: Prof. Esp. Alessandro de Menezes Fukuta

Franca

2026

RESUMO

O presente trabalho apresenta o desenvolvimento do aplicativo **Inteligente de Controle Financeiro Pessoal**, proposto na disciplina Laboratório de Desenvolvimento para Dispositivos Móveis do curso de Desenvolvimento de Software Multiplataforma da FATEC Dr. Thomaz Novelino. O projeto tem como tema o gerenciamento financeiro pessoal por meio de dispositivos móveis, partindo do problema da dificuldade que muitos indivíduos enfrentam para organizar receitas, despesas e metas financeiras de forma estruturada e visualmente compreensível. O objetivo principal consiste em desenvolver um aplicativo móvel híbrido capaz de registrar transações financeiras, categorizar gastos, definir orçamentos mensais e apresentar relatórios gráficos comparativos, auxiliando na tomada de decisões financeiras.

A metodologia adotada envolveu levantamento de requisitos, definição da arquitetura do sistema, modelagem do banco de dados relacional em PostgreSQL e estruturação de uma API REST para comunicação entre o aplicativo e o servidor. Foram utilizadas tecnologias como React Native no desenvolvimento do front-end móvel e Node.js com Express na construção do back-end, além de versionamento com Git e organização das atividades por meio de planejamento em sprint. A aplicação foi projetada para ser hospedada em ambiente de servidor VPS, garantindo controle de infraestrutura e possibilidade de escalabilidade.

Como resultado, foi definida uma estrutura de dados organizada em entidades como usuários, transações, categorias, metas e orçamentos, permitindo controle financeiro detalhado e geração de relatórios comparativos mensais. Observou-se que a modelagem adequada do banco de dados e a organização prévia do projeto contribuem significativamente para a eficiência no desenvolvimento e manutenção do sistema. Conclui-se que o uso de tecnologias multiplataforma e boas práticas de engenharia de software possibilita a construção de soluções acessíveis e funcionais para o controle financeiro pessoal, podendo futuramente ser expandido com novos recursos analíticos e integrações externas.

Palavras-chave: Controle financeiro. Aplicativo móvel. Banco de dados. API REST. Desenvolvimento multiplataforma

1 INTRODUÇÃO

A organização financeira pessoal é um dos principais desafios enfrentados por grande parte da população, especialmente diante do fácil acesso ao crédito, da multiplicidade de gastos recorrentes e da ausência de planejamento estruturado. A falta de controle sobre receitas e despesas pode gerar endividamento, desorganização orçamentária e dificuldades na tomada de decisões financeiras.

Com o avanço da tecnologia e a ampla utilização de dispositivos móveis, surgem oportunidades para o desenvolvimento de soluções digitais que auxiliem no gerenciamento financeiro de forma prática, acessível e intuitiva. Nesse contexto, o presente trabalho propõe o desenvolvimento do **Aplicativo Inteligente de Controle Financeiro Pessoal**, uma aplicação móvel híbrida voltada ao registro, acompanhamento e análise de movimentações financeiras.

O projeto será desenvolvido no âmbito da disciplina Laboratório de Desenvolvimento para Dispositivos Móveis, aplicando conceitos de modelagem de banco de dados, arquitetura de sistemas, desenvolvimento de API e boas práticas de engenharia de software.

2 OBJETIVO DO PROJETO

O objetivo geral deste projeto é desenvolver um aplicativo móvel híbrido para controle financeiro pessoal, permitindo que usuários registrem receitas e despesas, organizem seus gastos por categorias, definam metas financeiras e acompanhem relatórios comparativos mensais.

Como objetivos específicos, destacam-se:

- Estruturar uma base de dados relacional utilizando PostgreSQL;
- Desenvolver uma API REST para comunicação entre front-end e back-end;
- Implementar funcionalidades de cadastro e autenticação de usuários;
- Permitir a categorização e organização de transações financeiras;
- Disponibilizar relatórios gráficos para análise de desempenho financeiro;
- Aplicar práticas de versionamento e organização de projeto.

3 JUSTIFICATIVA

A educação financeira é um tema cada vez mais relevante na sociedade contemporânea. No entanto, muitas pessoas ainda apresentam dificuldades em manter controle efetivo sobre seus gastos e receitas. A ausência de planejamento financeiro pode resultar em endividamento, inadimplência e dificuldades na realização de objetivos pessoais.

Diante desse cenário, o desenvolvimento de um aplicativo de controle financeiro torna-se relevante por oferecer uma ferramenta acessível e prática, que auxilia o usuário na visualização clara de sua situação financeira. Além disso, o projeto contribui academicamente ao aplicar conhecimentos técnicos relacionados a desenvolvimento mobile, modelagem de banco de dados, arquitetura de sistemas e infraestrutura de servidores.

Portanto, o aplicativo proposto busca unir tecnologia e organização financeira, promovendo maior consciência e controle sobre as finanças pessoais.

4 PÚBLICO-ALVO

O público-alvo do aplicativo é composto por:

- Jovens adultos que desejam organizar suas finanças pessoais;
- Estudantes universitários que precisam controlar gastos mensais;
- Trabalhadores autônomos que necessitam registrar receitas e despesas;
- Pessoas que desejam definir metas financeiras e acompanhar sua evolução.

O aplicativo é voltado a usuários que buscam uma solução simples, prática e acessível para controle financeiro por meio de dispositivos móveis.

5 FUNCIONALIDADES PRINCIPAIS

O aplicativo contará com as seguintes funcionalidades principais:

- Cadastro e autenticação de usuários;

- Registro de receitas e despesas;
- Categorização de transações financeiras;
- Definição de metas financeiras;
- Controle de orçamento mensal;
- Visualização de relatórios gráficos;
- Comparativo de gastos por período;
- Gerenciamento de configurações do usuário.

6 MODELAGEM E BANCO DE DADOS

O sistema do Aplicativo Inteligente de Controle Financeiro Pessoal utiliza o Sistema Gerenciador de Banco de Dados **PostgreSQL**, escolhido por sua robustez, confiabilidade, desempenho e ampla utilização em aplicações modernas. O banco encontra-se hospedado em ambiente de servidor VPS, garantindo controle de infraestrutura e possibilidade de escalabilidade.

Para a manipulação e modelagem dos dados, foi utilizado o ORM Prisma, que permite a definição estruturada das entidades por meio de schema declarativo, facilitando a geração de migrations, integridade relacional e comunicação segura com a API desenvolvida em Node.js.

A modelagem foi construída seguindo o modelo relacional, com utilização de chaves primárias do tipo UUID, garantindo unicidade global dos registros.

6.1 **ESTRUTURA DAS ENTIDADES**

A base de dados foi organizada a partir das seguintes entidades principais:

Usuário (User)

Entidade central do sistema, responsável por armazenar informações cadastrais como nome, e-mail e senha. Cada usuário pode possuir múltiplas contas, categorias, transações, cartões de crédito, orçamentos e registros recorrentes.

Conta (Account)

Representa contas financeiras do usuário, como conta corrente, poupança ou carteira digital. Cada conta possui saldo, tipo e relacionamento direto com o usuário. As transações podem estar associadas como origem ou destino da movimentação.

Cartão de Crédito (CreditCard)

Armazena informações sobre cartões vinculados ao usuário, incluindo limite disponível, dia de fechamento e vencimento. Permite controle de compras parceladas e lançamentos específicos.

Categoria (Category)

Responsável pela organização das transações financeiras. Cada categoria pertence a um usuário e pode conter atributos visuais, como ícone e cor, facilitando a identificação no aplicativo.

Transação (Transaction)

Entidade responsável pelo registro de movimentações financeiras, podendo ser do tipo receita (INCOME), despesa (EXPENSE) ou transferência (TRANSFER). Cada transação pode estar vinculada a uma conta de origem, conta de destino, cartão de crédito ou categoria específica.

Parcela (Installment)

Permite o controle de compras parceladas, armazenando número da parcela, valor, data de vencimento e status de pagamento.

Orçamento (Budget)

Define limites financeiros por categoria em determinado mês e ano, possibilitando controle e comparação com os gastos realizados.

Transação Recorrente (RecurringTransaction)

Responsável por registrar receitas ou despesas que ocorrem de forma periódica, podendo ser diária, semanal, mensal ou anual. Essa estrutura permite automatizar lançamentos futuros no sistema.

6.2 **RELACIONAMENTOS**

O modelo segue o padrão de relacionamentos um-para-muitos (1:N), onde:

- Um usuário pode possuir várias contas, categorias e transações.
- Uma transação pode estar associada a uma conta, cartão ou categoria.
- Um orçamento está vinculado a uma categoria específica.
- Uma transação recorrente pode gerar múltiplas transações.

Essa organização garante integridade referencial e permite escalabilidade do sistema.

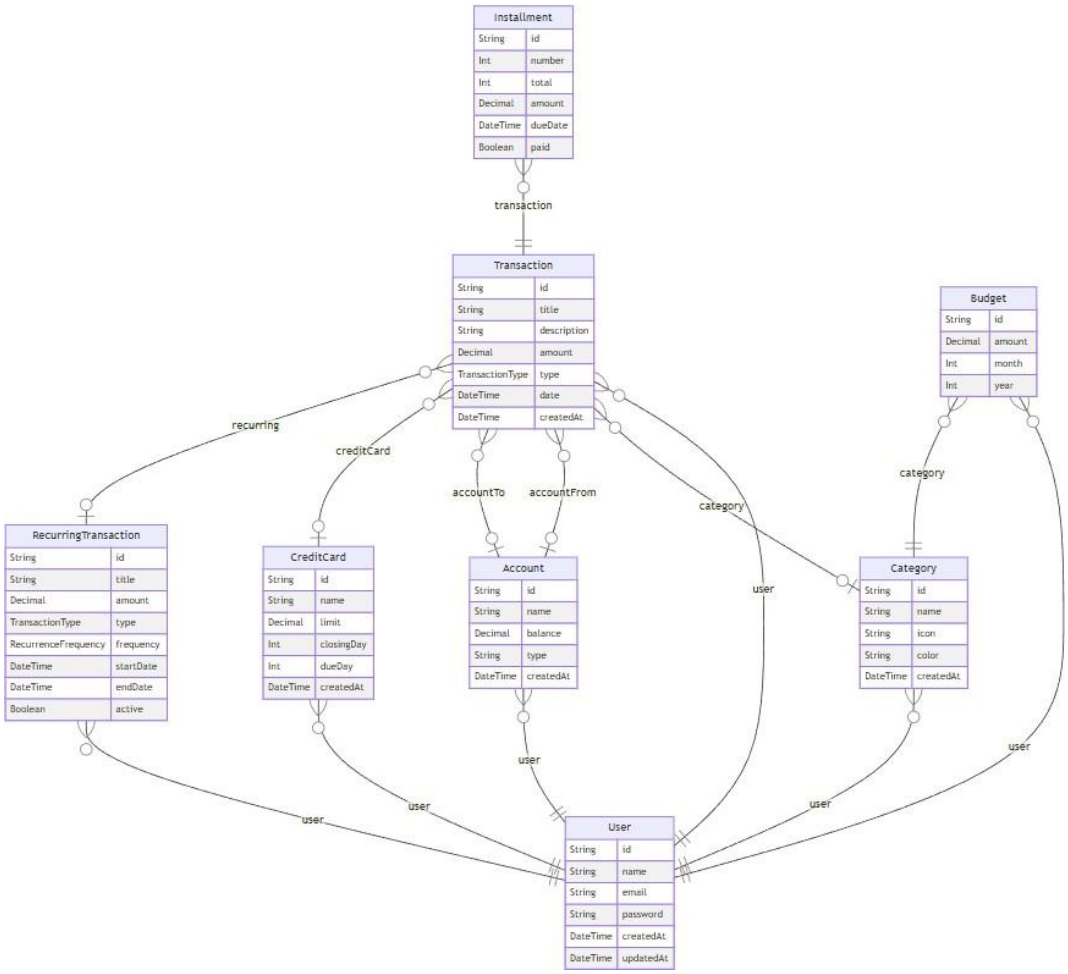

Figura 1 – Modelo de Entidade de Relacionamento do Banco de Dados

6.3 **JUSTIFICATIVA TÉCNICA**

A escolha do PostgreSQL deve-se à sua estabilidade, suporte a tipos avançados como decimal e DateTime, além de forte controle transacional. O uso do Prisma como ORM contribui para maior segurança no acesso aos dados, organização da camada de persistência e facilidade na manutenção futura do sistema.

Conclui-se que a modelagem adotada atende aos requisitos funcionais do aplicativo, permitindo controle financeiro detalhado, organização por categorias, gestão de cartões e automação de recorrências, mantendo a estrutura preparada para futuras expansões.

7 DESENVOLVIMENTO MOBILE

A aplicação mobile do sistema de Controle Financeiro Pessoal foi desenvolvida utilizando React Native, em conjunto com o framework Expo, permitindo a construção de aplicações multiplataforma capazes de executar em dispositivos Android e iOS a partir de uma única base de código.

A navegação do aplicativo foi estruturada utilizando Expo Router, que implementa um modelo de roteamento baseado em arquivos (file-based routing). Nesse modelo, cada arquivo dentro da pasta de rotas representa automaticamente uma tela da aplicação, simplificando a organização do código e a escalabilidade do projeto.

O aplicativo possui integração direta com a API do sistema, responsável por fornecer dados e executar operações de cadastro, consulta, atualização e exclusão de informações financeiras. A comunicação entre o aplicativo e o servidor ocorre por meio de requisições HTTP autenticadas utilizando JSON Web Token (JWT).

A estrutura do projeto foi organizada em camadas, separando responsabilidades entre interface do usuário, navegação, serviços de comunicação com a API e utilitários.

7.1 **ESTRUTURA DO PROJETO**

A organização do aplicativo segue uma estrutura modular que facilita a manutenção e evolução do sistema. As principais pastas da aplicação são:

app/

Contém todas as telas da aplicação e define a navegação principal do sistema. Essa pasta utiliza o padrão de roteamento baseado em arquivos do Expo Router.

lib/

Responsável pela camada de acesso à API, concentrando funções que realizam requisições ao backend. Cada arquivo representa um domínio específico do sistema, como autenticação, orçamento ou categorias.

components/

Armazena componentes reutilizáveis da interface, como botões, cartões informativos e elementos visuais utilizados em diferentes telas.

hooks/

Contém hooks personalizados utilizados para lógica compartilhada entre telas, incluindo controle de métricas e monitoramento de uso do aplicativo.

constants/

Define constantes utilizadas no sistema, como configurações visuais, cores e parâmetros reutilizados na interface.
Essa organização permite separar claramente a lógica de negócio, a camada de comunicação com o servidor e a interface do usuário.

7.2 **NAVEGAÇÃO DO APLICATIVO**

A navegação do sistema foi estruturada em dois fluxos principais: autenticação e área principal do usuário.

Ao iniciar o aplicativo, o usuário é direcionado para a tela inicial de autenticação, responsável pelo processo de login ou cadastro no sistema.

Após a autenticação bem-sucedida, o usuário é redirecionado para a área principal do aplicativo, organizada em um sistema de abas inferiores (tabs), permitindo acesso rápido às principais funcionalidades do sistema.

As principais telas disponíveis no aplicativo são:

Dashboard

Apresenta um resumo geral da situação financeira do usuário, incluindo saldo atual, receitas, despesas e movimentações recentes.

Transações

Permite visualizar e pesquisar movimentações financeiras registradas no sistema, incluindo receitas, despesas e transferências.

Cartão de Crédito

Responsável pelo gerenciamento de cartões cadastrados pelo usuário, exibindo limite total, limite utilizado e limite disponível.

Mais

Funciona como um hub de navegação, permitindo acesso a funcionalidades adicionais do sistema, como orçamento, categorias, relatórios e gerenciamento de recorrências.

Além dessas abas principais, o sistema possui telas secundárias acessadas por meio do menu “Mais”, incluindo gerenciamento de categorias, controle de orçamentos mensais, relatórios financeiros e controle de transações recorrentes.

7.3 **INTEGRAÇÃO COM O BACKEND**

A comunicação entre o aplicativo mobile e o backend é realizada por meio de uma camada centralizada de requisições localizada no diretório lib/api.ts.

Essa camada é responsável por configurar a URL base da API, enviar requisições HTTP e anexar automaticamente o token de autenticação no cabeçalho das requisições.

Após o processo de login, o token JWT recebido do servidor é armazenado localmente utilizando AsyncStorage, permitindo que o usuário permaneça autenticado durante o uso do aplicativo.

Entre os principais recursos consumidos pela aplicação estão:

- Autenticação de usuários
- Gerenciamento de categorias
- Controle de cartões de crédito
- Gerenciamento de orçamentos
- Controle de transações recorrentes
- Registro de métricas de uso do sistema

Essa arquitetura permite desacoplar a interface mobile da lógica de negócio do sistema, garantindo maior flexibilidade e escalabilidade da aplicação.

7.4 **MONITORAMENTO E MÉTRICAS DE USO**

O aplicativo implementa um sistema de telemetria responsável por registrar eventos de uso do sistema. Esse mecanismo permite coletar dados sobre a interação dos usuários com as funcionalidades do aplicativo.

Para isso, foi implementado um hook personalizado responsável por registrar automaticamente eventos como:

- abertura de telas;
- início e término de sessões;
- tempo de permanência em cada tela;
- cliques em botões e ações relevantes.

Esses dados são enviados para o backend por meio de endpoints específicos de métricas, possibilitando análises sobre comportamento de uso, desempenho de funcionalidades e experimentos A/B. O monitoramento contribui para a melhoria contínua da aplicação, permitindo identificar padrões de utilização e possíveis pontos de melhoria na experiência do usuário.

8 BACKEND E API DO SISTEMA

O backend do sistema de Controle Financeiro Pessoal foi desenvolvido utilizando Node.js, com o framework Express, adotando a linguagem TypeScript para garantir maior segurança de tipos e organização do código.
A API foi projetada seguindo o padrão REST (Representational State Transfer), permitindo a comunicação entre a aplicação mobile e o servidor por meio de requisições HTTP estruturadas.

O sistema utiliza PostgreSQL como banco de dados relacional, acessado por meio do Prisma ORM, responsável por mapear as entidades do banco para objetos manipuláveis na aplicação.

Além disso, o backend implementa um sistema de autenticação baseado em JSON Web Token (JWT), garantindo que apenas usuários autenticados possam acessar recursos protegidos da API.

A arquitetura da aplicação foi organizada em camadas, separando responsabilidades entre rotas, controladores, serviços e acesso ao banco de dados, o que facilita a manutenção e escalabilidade do sistema.

8.1 ESTRUTURA DO PROJETO

A estrutura do backend foi organizada de forma modular, separando os diferentes níveis da aplicação.

Os principais diretórios do projeto são:

routes/

Responsável pela definição das rotas da API. Cada arquivo agrupa endpoints relacionados a um domínio específico do sistema, como usuários, categorias ou transações.

controllers/

Contém os controladores responsáveis por receber as requisições HTTP, validar os dados de entrada e retornar as respostas apropriadas ao cliente.

services/

Camada responsável pela implementação da lógica de negócio do sistema. Os serviços executam as regras da aplicação e realizam a comunicação com o banco de dados por meio do Prisma.

interfaces/

Define contratos de dados utilizados na aplicação, garantindo padronização nas estruturas de entrada e saída entre as camadas do sistema.

middlewares/

Contém funções intermediárias executadas durante o processamento das requisições, como o middleware responsável pela validação do token de autenticação.

prisma/

Contém o arquivo de definição do modelo de dados (schema.prisma), utilizado pelo Prisma ORM para gerar o cliente de acesso ao banco de dados e gerenciar migrations.

Essa organização em camadas permite separar claramente a responsabilidade de cada componente do sistema, contribuindo para maior organização e reutilização de código.

8.2 **AUTORIZAÇÃO E SEGURANÇA**

O sistema implementa autenticação baseada em JSON Web Token (JWT), garantindo que apenas usuários autorizados possam acessar recursos protegidos da API.

O processo de autenticação ocorre por meio do endpoint de login, no qual o usuário fornece seu e-mail e senha cadastrados. A senha é comparada com o hash armazenado no banco de dados utilizando a biblioteca bcryptjs.

Caso as credenciais sejam válidas, o sistema gera um token JWT contendo informações do usuário autenticado. Esse token possui tempo de expiração definido e deve ser enviado pelo cliente em todas as requisições protegidas por meio do cabeçalho HTTP:

Authorization: Bearer <token>

O backend possui um middleware responsável por validar esse token. Caso o token seja inválido ou inexistente, o sistema retorna uma resposta HTTP com código 401 (Unauthorized).

Quando o token é validado com sucesso, as informações do usuário são adicionadas à requisição, permitindo que os controladores identifiquem o usuário responsável pela operação.

8.3 **DOMÍNIOS DA API**

A API do sistema foi organizada em diferentes domínios funcionais, cada um responsável por gerenciar uma parte específica da aplicação.
Entre os principais módulos da API estão:

Autenticação
Responsável pelo processo de login dos usuários e geração de tokens de acesso.

Usuários
Permite o cadastro, consulta, atualização e remoção de usuários no sistema.

Contas
Gerencia contas financeiras associadas ao usuário, como contas correntes, poupanças ou carteiras digitais.

Categorias
Responsável pela criação e organização das categorias utilizadas para classificar receitas e despesas.

Transações
Gerencia as movimentações financeiras do usuário, incluindo receitas, despesas e transferências entre contas.

Parcelas
Permite o controle de compras parceladas, armazenando informações sobre número da parcela, valor e data de vencimento.

Orçamentos
Define limites de gastos por categoria em períodos específicos, permitindo monitoramento financeiro.

Cartões de Crédito
Gerencia cartões cadastrados pelo usuário, incluindo limite disponível e controle de gastos.

Transações Recorrentes
Permite registrar receitas ou despesas que ocorrem de forma periódica, como assinaturas ou salários.

Essa divisão por domínios facilita a manutenção da aplicação e possibilita evolução independente de cada módulo do sistema.

8.4 **EXPERIMENTOS E MÉTRICAS**

O backend também implementa um sistema de coleta de métricas e experimentação que permite analisar o comportamento dos usuários dentro da aplicação.

Esse mecanismo registra eventos gerados pelo aplicativo mobile, como visualização de telas, cliques em botões e duração de sessões.

Os eventos são armazenados no banco de dados e posteriormente utilizados para gerar análises e relatórios estatísticos.

Entre as funcionalidades disponíveis nesse módulo estão:

- registro de eventos de interação do usuário;
- análise de taxa de cliques (CTR) em experimentos;
- cálculo de tempo médio de uso do aplicativo;
- geração de resumos estatísticos por variante de experimento.

O sistema também permite a criação de experimentos A/B, nos quais diferentes versões de uma funcionalidade podem ser atribuídas a grupos distintos de usuários. Essa abordagem possibilita avaliar qual variante apresenta melhor desempenho em termos de engajamento ou utilização

8.5 **DOCUMENTAÇÃO DA API**

Para facilitar o desenvolvimento e integração com outras aplicações, o backend disponibiliza documentação interativa da API utilizando Swagger UI, acessível por meio do endpoint:

/docs

Essa interface permite visualizar todos os endpoints disponíveis, incluindo métodos HTTP, parâmetros, exemplos de requisição e respostas esperadas.

Além disso, o sistema também disponibiliza uma página HTML estática com documentação simplificada da API, acessível pelo endpoint:

/documentacao

Esses recursos auxiliam no processo de desenvolvimento e integração entre diferentes partes do sistema.

8.6 **TESTES AUTOMATIZADOS**

O backend possui um conjunto de testes automatizados implementados utilizando Jest e Supertest.

Esses testes são responsáveis por validar o funcionamento dos principais endpoints da API, garantindo que as rotas respondam corretamente às requisições e que as regras de negócio sejam executadas conforme esperado.

Entre os módulos cobertos pelos testes estão:

- autenticação de usuários;
- gerenciamento de usuários;
- contas financeiras;
- categorias;
- transações;
- parcelas;
- orçamentos;
- cartões de crédito;
- transações recorrentes;
- experimentos;
- métricas;
- configurações do sistema.

A utilização de testes automatizados contribui para a confiabilidade do sistema, permitindo identificar falhas rapidamente durante o processo de desenvolvimento.

9 PAINEL WEB ADMINISTRATIVO

Além da aplicação mobile, o sistema também possui um painel web administrativo desenvolvido utilizando Next.js, framework baseado em React voltado para aplicações web modernas.

O painel tem como objetivo fornecer uma interface simples para administração do sistema, permitindo visualizar métricas de uso da aplicação e gerenciar configurações globais do aplicativo.

A aplicação foi desenvolvida utilizando TypeScript para garantir maior segurança no desenvolvimento e Tailwind CSS para estilização da interface.

Diferentemente do aplicativo mobile, o frontend web possui escopo reduzido e é voltado principalmente para funcionalidades administrativas.

9.1 FUNCIONALIDADES DO PAINEL

O painel administrativo possui três funcionalidades principais:

Autenticação
O acesso ao painel é protegido por um mecanismo simples de autenticação baseado em senha. Após o login bem-sucedido, uma sessão é criada por meio de cookies seguros, permitindo acesso às demais páginas do sistema.

Dashboard de Métricas
O painel disponibiliza um dashboard analítico que apresenta métricas de utilização do aplicativo, incluindo quantidade de eventos registrados, número de usuários ativos, visualizações de tela e interações realizadas pelos usuários.

Essas informações são apresentadas por meio de gráficos e indicadores que facilitam a análise do comportamento de uso da aplicação.

Gerenciamento de Configurações
O sistema também permite editar configurações globais do aplicativo, como nome do sistema, descrição e versão. Essas informações podem ser atualizadas diretamente pelo painel e são utilizadas pelo aplicativo mobile.

9.2 INTEGRAÇÃO COM O BACKEND

O frontend web se comunica com o backend por meio de requisições HTTP para a API do sistema.

Algumas rotas são intermediadas por endpoints internos do próprio Next.js, que funcionam como proxy para o backend e permitem validar a sessão do usuário antes de encaminhar a requisição.

Entre os principais recursos consumidos pelo painel estão:

- consulta de métricas do sistema;
- gerenciamento de configurações globais da aplicação.

Essa integração permite centralizar o controle administrativo do sistema em uma interface web simples e acessível.

10 CONCLUSÃO

O desenvolvimento do sistema de Controle Financeiro Pessoal Inteligente possibilitou a criação de uma solução completa voltada ao gerenciamento financeiro individual, integrando tecnologias modernas de desenvolvimento de software, banco de dados e aplicações móveis. O projeto foi estruturado com o objetivo de oferecer aos usuários uma ferramenta prática e eficiente para registrar, organizar e analisar suas finanças pessoais.

Ao longo do desenvolvimento, foi possível aplicar conceitos fundamentais de engenharia de software, como modelagem de dados, arquitetura em camadas, separação de responsabilidades e integração entre diferentes plataformas. A construção do sistema envolveu três componentes principais: o backend responsável pela lógica de negócio e persistência de dados, o aplicativo mobile voltado para interação direta com o usuário, e um painel web administrativo destinado ao monitoramento e configuração do sistema.

A modelagem do banco de dados foi realizada utilizando o modelo relacional no PostgreSQL, com o suporte do Prisma ORM, permitindo definir de forma clara as entidades e seus relacionamentos. A utilização de identificadores globais do tipo UUID contribuiu para garantir unicidade e maior segurança na manipulação dos registros. Essa estrutura

possibilitou representar de maneira adequada os elementos centrais do sistema, como usuários, contas, categorias, transações, cartões de crédito, orçamentos e recorrências.

No backend, desenvolvido com Node.js, Express e TypeScript, foi implementada uma arquitetura organizada em camadas, separando rotas, controladores e serviços. Essa abordagem contribuiu para melhorar a organização do código e facilitar a manutenção e evolução da aplicação. Além disso, foi implementado um sistema de autenticação baseado em JSON Web Token (JWT), garantindo segurança no acesso às funcionalidades da API.

A aplicação mobile, desenvolvida com React Native e Expo, representa a principal interface de interação do usuário com o sistema. Por meio dela, é possível registrar movimentações financeiras, acompanhar gastos, controlar cartões de crédito, definir orçamentos e visualizar informações consolidadas sobre a situação financeira pessoal. A estrutura baseada em navegação por abas e organização modular das telas contribuiu para proporcionar uma experiência de uso simples e intuitiva.

Outro aspecto relevante do projeto foi a implementação de um sistema de telemetria e métricas de uso, capaz de registrar eventos de interação do usuário dentro do aplicativo. Esses dados permitem analisar padrões de utilização do sistema e possibilitam a realização de experimentos A/B, recurso frequentemente utilizado em aplicações modernas para avaliar melhorias de interface ou funcionalidades.

Complementando o ecossistema do sistema, foi desenvolvido um painel web administrativo, utilizando Next.js, que permite visualizar métricas de uso da aplicação e gerenciar configurações globais do sistema. Embora possua escopo reduzido em comparação ao aplicativo mobile, esse painel oferece uma forma prática de monitorar o funcionamento da plataforma.

Durante o desenvolvimento do projeto, diversos desafios foram enfrentados, principalmente relacionados à integração entre diferentes tecnologias, organização da arquitetura do sistema e garantia de comunicação eficiente entre frontend, backend e banco de dados. A adoção de boas práticas de desenvolvimento, juntamente com a utilização de ferramentas modernas como TypeScript, Prisma, testes automatizados e documentação da API, contribuiu para aumentar a qualidade e confiabilidade do sistema.

Como resultado, foi possível construir uma aplicação funcional capaz de auxiliar usuários no acompanhamento de suas finanças pessoais, fornecendo recursos de organização, análise e planejamento financeiro. O sistema apresenta potencial de evolução, podendo receber melhorias futuras, como integração com instituições financeiras, geração de relatórios mais avançados, notificações inteligentes e recursos adicionais de análise de dados.

Dessa forma, conclui-se que os objetivos propostos para o projeto foram alcançados, resultando em uma solução tecnológica consistente que demonstra, na prática, a aplicação dos conhecimentos adquiridos ao longo da formação acadêmica nas áreas de desenvolvimento de software, bancos de dados, arquitetura de sistemas e desenvolvimento de aplicações móveis.

REFERÊNCIAS

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL Documentation**. Disponível em: https://www.postgresql.org/docs/. Acesso em: 15 fev. 2026.

PRISMA. **Prisma ORM Documentation**. Disponível em: https://www.prisma.io/docs. Acesso em: 15 fev. 2026.

NODE.JS FOUNDATION. **Node.js Documentation**. Disponível em: https://nodejs.org/en/docs. Acesso em: 15 fev. 2026.

EXPRESS.JS. **Express – Fast, unopinionated, minimalist web framework for Node.js**. Disponível em: https://expressjs.com/. Acesso em: 15 fev. 2026.

META PLATFORMS, INC. **React Native Documentation**. Disponível em: https://reactnative.dev/docs/getting-started. Acesso em: 15 fev. 2026.

FIELDING, Roy Thomas. **Architectural Styles and the Design of Network-based Software Architectures**. 2000. Tese (Doutorado em Ciência da Computação) – University of California, Irvine, 2000. Disponível em: https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm. Acesso em: 15 fev. 2026.

GITHUB. **GitHub Actions Documentation**. Disponível em: https://docs.github.com/en/actions. Acesso em: 15 fev. 2026.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 6. ed. Rio de Janeiro: Elsevier, 2012.